

# Codex Community Hackathon

## 20-Day Preparation Plan

---

SUST CSE Carnival 2026

bKash presents | In association with Codex & Poridhi

### 3-Member Team

Target: **Top 40** in Online Preliminary

Phase 2 — 3-Hour Online Qualification Round

AI & API Hackathon — Official Preparation Guide

June 15, 2026

## Contents

---

|          |                                                                                 |          |
|----------|---------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Rulebook Key Takeaways</b>                                                   | <b>2</b> |
| <b>2</b> | <b>What the Judges Want (Decoded from Rulebook)</b>                             | <b>2</b> |
| <b>3</b> | <b>Team Role Assignment (3 Members)</b>                                         | <b>2</b> |
| <b>4</b> | <b>20-Day Preparation Schedule</b>                                              | <b>3</b> |
| 4.1      | Phase A: Foundation (Days 1–5) — “Learn the Tools” . . . . .                    | 3        |
| 4.1.1    | Day 1–2: Understand the Battlefield . . . . .                                   | 3        |
| 4.1.2    | Day 3: Tech Stack Lock-In . . . . .                                             | 3        |
| 4.1.3    | Day 4–5: API Speed Runs . . . . .                                               | 3        |
| 4.2      | Phase B: Vibe Coding Mastery (Days 6–10) — “Code at the Speed of Thought” . . . | 4        |
| 4.2.1    | Day 6–7: Prompt Engineering Bootcamp . . . . .                                  | 4        |
| 4.2.2    | Day 8–9: Vibe Coding Drills (Timed) . . . . .                                   | 4        |
| 4.2.3    | Day 10: Puku Editor & CLI Familiarization . . . . .                             | 4        |
| 4.3      | Phase C: Mock Hackathons (Days 11–16) — “Simulate the Pressure” . . . . .       | 5        |
| 4.3.1    | Day 11–12: Mock Hackathon #1 (3 hours) . . . . .                                | 5        |
| 4.3.2    | Day 13–14: Mock Hackathon #2 (3 hours) . . . . .                                | 5        |
| 4.3.3    | Day 15–16: Mock Hackathon #3 (3 hours) — Full Dress Rehearsal . . . . .         | 5        |
| 4.4      | Phase D: Sharpening (Days 17–19) — “Eliminate Friction” . . . . .               | 5        |
| 4.4.1    | Day 17: Build Your Arsenal . . . . .                                            | 5        |
| 4.4.2    | Day 18: Speed Optimization . . . . .                                            | 6        |
| 4.4.3    | Day 19: Mental Models & Strategy . . . . .                                      | 6        |
| 4.5      | Phase E: Competition Day (Day 20) — “Execute” . . . . .                         | 6        |
| 4.5.1    | Pre-Competition Checklist . . . . .                                             | 6        |
| 4.5.2    | During the 3-Hour Preliminary — Execution Protocol . . . . .                    | 7        |
| <b>5</b> | <b>Practice Problem Bank</b>                                                    | <b>7</b> |
| <b>6</b> | <b>Critical Success Factors</b>                                                 | <b>8</b> |
| <b>7</b> | <b>Pre-Competition Allowed Materials (per Rulebook)</b>                         | <b>8</b> |
| <b>8</b> | <b>Prompt Engineering Cheat Sheet (For the Hackathon)</b>                       | <b>8</b> |

## 1. Rulebook Key Takeaways

### ! IMPORTANT

**Your immediate goal:** Pass **Phase 2 — Online Preliminary Round** (3 hours) to qualify as one of the **top 40 teams** for the 24-hour on-site hackathon.

| Aspect                  | Details                                                                |
|-------------------------|------------------------------------------------------------------------|
| <b>Phase 1</b>          | Mock Preliminary (Online) — No elimination, just familiarization       |
| <b>Phase 2</b>          | 3-hour Online Preliminary — <b>THIS IS THE GATE</b> → Top 40 selected  |
| <b>Phase 3</b>          | 24-hour On-Site Hackathon at SUST                                      |
| <b>Team Size</b>        | 1–3 members (you have 3 ✓)                                             |
| <b>What's Built</b>     | Full-stack (Web/Android/Both) with <b>AI + API integrations</b>        |
| <b>AI Tools</b>         | Allowed for suggestions/debugging — <b>core logic must be your own</b> |
| <b>Pre-written Code</b> | ✗ NOT allowed — but wireframes/planning docs ✓ ARE allowed             |
| <b>APIs Provided</b>    | OpenAI API Credits (from Codex) + Poridhi Puku Editor/CLI              |
| <b>Judging Focus</b>    | Process understanding, efficiency, scalability, creative AI use        |

## 2. What the Judges Want (Decoded from Rulebook)

Based on the sample problem and evaluation criteria, here's what will get you into the top 40:

- Multi-API Orchestration** — Chain multiple APIs together (STT → Translation → LLM → TTS)
- Creative AI Integration** — Not just “call GPT” but structured prompt pipelines with reasoning
- Full-Stack Functional Product** — Working UI + Backend + AI, not just a backend script
- Real-World Problem Alignment** — Solutions must address the announced scenario meaningfully
- Speed of Execution** — 3 hours is brutally short; you need muscle memory, not learning

## 3. Team Role Assignment (3 Members)

### 💡 TIP

Assign roles **NOW** and practice in these roles for 20 days. Don't switch during the competition.

| Role                                                     | Responsibility                                                            | Focus During Prelim                                                 |
|----------------------------------------------------------|---------------------------------------------------------------------------|---------------------------------------------------------------------|
| 🧠 <b>Member 1:</b><br><b>Architect + AI/Backend</b>      | System design, API integrations, prompt engineering, LLM pipelines        | Designs the solution architecture in first 10 min, builds API layer |
| 🎨 <b>Member 2:</b><br><b>Frontend + UI/UX</b>            | Rapid UI building, component library, connecting frontend to APIs         | Spins up the UI shell immediately, connects to backend APIs         |
| 🔧 <b>Member 3:</b><br><b>Full-Stack Floater + DevOps</b> | Fills gaps, handles data processing, OCR/ML pipelines, deployment, README | Handles secondary APIs, writes documentation, tests end-to-end      |

## 4. 20-Day Preparation Schedule

---

### 4.1 Phase A: Foundation (Days 1–5) — “Learn the Tools”

#### 4.1.1 Day 1–2: Understand the Battlefield

- ☐ **All 3 members** read the rulebook thoroughly ✓
- ☐ Analyze the sample problem scenario in detail — break it into sub-tasks
- ☐ List every API/service mentioned: OpenAI, Google STT/TTS, Google Cloud Vision, Tesseract, Azure Form Recognizer, DeepL, ElevenLabs, AssemblyAI, Whisper, Hugging Face
- ☐ Each member signs up for free tiers of **at least 5 APIs** and gets API keys ready
- ☐ Set up a shared credentials vault (use a private `.env` file template)
- ☐ Create a shared GitHub org/repo for practice projects

#### 4.1.2 Day 3: Tech Stack Lock-In

- ☐ **Decide and lock your stack** — no more deliberation after today:

| Layer             | Recommended Choice                 | Why                                               |
|-------------------|------------------------------------|---------------------------------------------------|
| <b>Frontend</b>   | Next.js or Vite + React            | Fast scaffolding, SSR optional, rich ecosystem    |
| <b>CSS</b>        | Tailwind CSS                       | Rapid prototyping, consistent design in minutes   |
| <b>Backend</b>    | FastAPI (Python) or Express.js     | Python: better AI/ML library support              |
| <b>AI/LLM</b>     | OpenAI API (GPT-4)                 | You'll get credits from Codex — master this first |
| <b>Database</b>   | Supabase or Firebase               | Instant setup, real-time, auth built-in           |
| <b>Deployment</b> | Vercel + Railway/Render            | One-click deploys                                 |
| <b>Voice</b>      | Whisper API + ElevenLabs/TTS       | Based on sample problem pattern                   |
| <b>OCR</b>        | Google Cloud Vision / Tesseract.js | Document digitization is a recurring theme        |

- ☐ Set up a **project template repo** with your chosen stack (boilerplate only — no problem-specific code!)
- ☐ Include: folder structure, `.env.example`, `package.json/requirements.txt`, basic routing, API client wrappers

#### 4.1.3 Day 4–5: API Speed Runs

- ☐ **Each member picks 3–4 APIs and builds a mini-project with each:**
  - Member 1: OpenAI Chat Completions, Function Calling, Structured Output, Embeddings
  - Member 2: Google Maps API, Firebase Auth, Supabase CRUD, any Chart/Viz library
  - Member 3: Google Cloud Vision OCR, Whisper STT, Google TTS, Translation APIs
- ☐ Document each API's **request/response format, latency, rate limits, error handling**
- ☐ Create a shared **API Cheat Sheet** with copy-paste code snippets

## 4.2 Phase B: Vibe Coding Mastery (Days 6–10) — “Code at the Speed of Thought”

### ! IMPORTANT

**Vibe coding** = using AI assistants (Copilot, Cursor, ChatGPT, Puku) to generate code rapidly while you steer architecture and logic. This is your **unfair advantage** — master it.

#### 4.2.1 Day 6–7: Prompt Engineering Bootcamp

- ❑ Practice crafting prompts for these common patterns:

##### Prompt Template Library (Build These)

###### 1. System Architecture Prompt

“Given this problem: [PROBLEM]. Design a full-stack architecture with: Frontend: [STACK], Backend: [STACK], APIs needed: list with justification, Data flow diagram, Database schema”

###### 2. API Integration Prompt

“Write a [LANGUAGE] function that: Calls [API\_NAME] with [INPUT], Handles errors (rate limit, timeout, auth failure), Returns structured output as [FORMAT], Include retry logic with exponential backoff”

###### 3. UI Component Prompt

“Create a React component for [DESCRIPTION] with: Responsive design (mobile-first), Loading states and error states, Accessibility (ARIA labels), Styled with Tailwind CSS, Accept props: [LIST]”

###### 4. LLM Pipeline Prompt

“You are a [ROLE]. Given the following [INPUT\_TYPE]: [INPUT]. Analyze and return a JSON object with: {field1, field2, ...}. Reasoning: provide step-by-step analysis before the final output.”

- ❑ Practice **prompt chaining** — output of one prompt feeds into the next
- ❑ Practice **few-shot prompting** — give examples of desired output
- ❑ Learn to use **structured output** (JSON mode) from OpenAI to avoid parsing headaches

#### 4.2.2 Day 8–9: Vibe Coding Drills (Timed)

- ❑ **Drill 1** (45 min): Build a full CRUD app with auth using only AI-assisted coding
- ❑ **Drill 2** (45 min): Build an API-chaining pipeline (Image → OCR → LLM Analysis → Report)
- ❑ **Drill 3** (45 min): Build a voice-enabled app (STT → Process → TTS)
- ❑ **Drill 4** (45 min): Build a data visualization dashboard from an API source
- ❑ After each drill, **review what slowed you down** and create shortcuts/snippets

#### 4.2.3 Day 10: Puku Editor & CLI Familiarization

- ❑ Sign up and explore **Poridhi’s Puku Editor and CLI** (provided during the hackathon)
- ❑ Understand its capabilities — it’s an “unlimited AI coding assistant”
- ❑ Practice using it alongside your normal IDE workflow
- ❑ Identify how Puku complements vs. overlaps with GitHub Copilot/ChatGPT

### 4.3 Phase C: Mock Hackathons (Days 11–16) — “Simulate the Pressure”

#### ⚠ CAUTION

This phase is **NON-NEGOTIABLE**. If you skip this, you will panic during the real preliminary and waste precious time.

#### 4.3.1 Day 11–12: Mock Hackathon #1 (3 hours)

- ☐ Pick a **sample problem** (use the healthcare one from the rulebook or create one)
- ☐ Run a full **3-hour simulation**:
  - **0:00–0:10** → Read problem, discuss, assign tasks, sketch architecture
  - **0:10–0:30** → Member 1: backend + API keys; Member 2: scaffold frontend; Member 3: repo + DB
  - **0:30–1:30** → Core feature development (each member on their assigned module)
  - **1:30–2:15** → Integration — connect all pieces, test end-to-end
  - **2:15–2:45** → Polish UI, fix bugs, add error handling
  - **2:45–3:00** → Final commit, write README, prepare submission
- ☐ **Post-mortem**: What went wrong? Where did you waste time? What was the bottleneck?

#### 4.3.2 Day 13–14: Mock Hackathon #2 (3 hours)

- ☐ Use a **different problem scenario** — create your own:
  - “*AI-Powered Smart Agriculture Advisory*”
  - “*AI-Based Urban Traffic Optimization Dashboard*”
  - “*Intelligent Disaster Response Coordination System*”
- ☐ Apply lessons from Mock #1
- ☐ Time each phase and **track improvement**
- ☐ Focus on **integration speed** — this is where most teams fail

#### 4.3.3 Day 15–16: Mock Hackathon #3 (3 hours) — Full Dress Rehearsal

- ☐ Simulate as close to real conditions as possible:
  - Use the same laptops you’ll use on the day
  - Same network setup (hotspot if needed)
  - No pausing, no breaks (just like the real thing)
- ☐ Have someone **invent a surprise problem** you haven’t seen
- ☐ Practice the **first 10 minutes** obsessively — this sets the pace for everything

### 4.4 Phase D: Sharpening (Days 17–19) — “Eliminate Friction”

#### 4.4.1 Day 17: Build Your Arsenal

- ☐ Create a **Starter Kit Document** (wireframes + planning docs are allowed!):
  - Common UI layouts (dashboard, form-heavy, card-based)

- Architecture diagrams for common patterns (monolith, microservices-lite)
  - Database schema templates (users, data objects, logs)
  - API integration patterns (auth flow, webhook handling, polling)
- ☐ Prepare **prompt templates** saved in a doc you can copy-paste from
  - ☐ Create a “**First 10 Minutes**” **checklist**:

- ☐ Read problem (2 min)
- ☐ Discuss approach (3 min)
- ☐ Sketch architecture on paper (3 min)
- ☐ Assign tasks + create repo (2 min)

#### 4.4.2 Day 18: Speed Optimization

- ☐ Practice **typing speed** — seriously, every second counts
- ☐ Set up **code snippets** in your IDE for common patterns:
  - API fetch wrapper
  - React component boilerplate
  - Express/FastAPI route boilerplate
  - Error handling wrapper
  - `.env` loading
- ☐ Set up **Git aliases** for rapid commits (`git ac "message" = add + commit`)
- ☐ Pre-configure your IDE with all extensions, themes, and settings
- ☐ Test your internet speed; prepare mobile hotspot as backup

#### 4.4.3 Day 19: Mental Models & Strategy

- ☐ Develop a **problem decomposition framework**:

1. Who is the user? → Defines UI/UX priorities
2. What data flows in/out? → Defines APIs needed
3. Where does AI add value? → Defines LLM integration points
4. What's the MVP? → Defines what to build in 3 hours
5. What's the “wow factor”? → Defines what makes you stand out

- ☐ Practice **rapid architecture whiteboarding** — 5 min to design a system for any problem
- ☐ Review all API cheat sheets one more time
- ☐ Review all prompt templates

### 4.5 Phase E: Competition Day (Day 20) — “Execute”

#### 4.5.1 Pre-Competition Checklist

- ☐ Laptop fully charged + charger packed
- ☐ All API keys verified and working (test each one)
- ☐ Starter Kit Document printed or easily accessible

- ☐ Prompt templates ready to copy-paste
- ☐ IDE configured with all extensions and snippets
- ☐ Git configured with SSH keys
- ☐ Mobile data ready as backup internet
- ☐ Multiplug/power strip packed
- ☐ Water and snacks ready

#### 4.5.2 During the 3-Hour Preliminary — Execution Protocol

| Time             | Action                                                                         | Who                |
|------------------|--------------------------------------------------------------------------------|--------------------|
| <b>0:00–0:02</b> | Read problem silently                                                          | All                |
| <b>0:02–0:05</b> | Discuss: identify users, data flow, AI integration points                      | All                |
| <b>0:05–0:10</b> | Sketch architecture, assign tasks, create GitHub repo                          | Architect          |
| <b>0:10–0:15</b> | Scaffold: <code>npx create-...</code> , install deps, set up <code>.env</code> | All (own modules)  |
| <b>0:15–1:00</b> | <b>Core Build Sprint 1:</b> Build modules independently                        | All (parallel)     |
| <b>1:00–1:15</b> | <b>Sync Check:</b> Quick standup — done? blocked?                              | All                |
| <b>1:15–2:00</b> | <b>Core Build Sprint 2:</b> Integration begins                                 | All                |
| <b>2:00–2:30</b> | <b>Integration &amp; Testing:</b> End-to-end flow                              | All                |
| <b>2:30–2:50</b> | <b>Polish:</b> UI cleanup, error handling, loading states                      | Frontend + Floater |
| <b>2:50–3:00</b> | <b>Submission:</b> Final commit, README, submit                                | Floater leads      |

## 5. Practice Problem Bank

Create your own problems following the rulebook’s pattern (real-world scenario + multi-task + AI + APIs):

| # | Problem                                            | Key APIs                                                |
|---|----------------------------------------------------|---------------------------------------------------------|
| 1 | AI-Powered Rural Healthcare Triage (from rulebook) | STT, Translation, OCR, LLM, TTS, Anomaly Detection      |
| 2 | Smart Agriculture Advisory for Farmers             | Weather API, Satellite/Image API, LLM, Translation, TTS |
| 3 | Urban Traffic Optimization Dashboard               | Maps API, Real-time data, LLM analysis, Visualization   |
| 4 | Disaster Response Coordination System              | Geolocation, Messaging API, LLM triage, Maps            |
| 5 | AI Education Tutor for Rural Schools               | STT, Translation, LLM, TTS, Quiz Generation             |
| 6 | Supply Chain Anomaly Detection Platform            | Data APIs, LLM analysis, Visualization, Alerting        |

## 6. Critical Success Factors



**⚠ WARNING****Top reasons teams FAIL the preliminary:**

1. **Time mismanagement** — spending 45 min on architecture discussion instead of 10
2. **API setup failures** — API key doesn't work, wrong endpoint, CORS issues
3. **Integration hell** — each member builds their part but they don't connect
4. **Ugly UI** — judges see the product first; if it looks bad, they assume it IS bad
5. **No README** — judges can't run your project = instant disadvantage

**💡 TIP****Top reasons teams WIN:**

1. **Muscle memory** — they've done this 3+ times before in practice
2. **Seamless API chaining** — multiple APIs working together in a pipeline
3. **Creative AI use** — not just "send text to GPT" but structured reasoning with JSON output
4. **Polished UI** — even a simple but clean UI beats a complex but ugly one
5. **Clear README** — setup instructions that actually work

## 7. Pre-Competition Allowed Materials (per Rulebook)

**You CAN bring:**

- ✓ Wireframes and UI mockups
- ✓ Planning documents and architecture diagrams
- ✓ Prompt template libraries
- ✓ API cheat sheets with code patterns
- ✓ This preparation checklist

**You CANNOT bring:**

- ✗ Any pre-written source code
- ✗ Pre-built components or modules
- ✗ Any project developed before the event

## 8. Prompt Engineering Cheat Sheet (For the Hackathon)

### Pattern 1: System Prompt for Domain Expert LLM

```
You are an expert [DOMAIN] analyst. Given the following [INPUT_TYPE],  
analyze it and provide:
```

- ```
1. [ANALYSIS_POINT_1]  
2. [ANALYSIS_POINT_2]  
3. [ANALYSIS_POINT_3]
```

```
Return your response as a JSON object with the following schema:
```

```
{  
  "analysis": string,  
  "score": number (1-10),  
  "recommendations": string[],  
  "severity": "low" | "medium" | "high" | "critical"  
}
```

Think step-by-step before providing your final answer.

## Pattern 2: Multi-Step Reasoning Chain

Step 1: Extract key entities from the input  
Step 2: Cross-reference with known patterns  
Step 3: Generate risk assessment  
Step 4: Provide actionable recommendations

Input: [USER\_INPUT]

Context: [ADDITIONAL\_CONTEXT]

## Pattern 3: Few-Shot with Examples

Here are examples of correct analysis:

Example 1:

Input: [EXAMPLE\_INPUT\_1]

Output: [EXAMPLE\_OUTPUT\_1]

Example 2:

Input: [EXAMPLE\_INPUT\_2]

Output: [EXAMPLE\_OUTPUT\_2]

Now analyze this:

Input: [ACTUAL\_INPUT]

### NOTE

**Remember:** The preliminary is only 3 hours. You're not building a production app — you're building a **convincing, functional prototype** that demonstrates creative AI + API integration. Prioritize **breadth** (touch all tasks) over **depth** (perfect one task).

Good Luck! 🚀

Prepared for SUST CSE Carnival 2026 — Codex Community Hackathon